Báo cáo kiểm tra giữa kỳ Lập trình Python

Sinh viên: Nguyễn Duy Trường | **MSSV:** 24150180

Trường: Đại học Gia Định | **Lớp học phần:** 020100138002

Học kỳ: 1 - Năm học 2026-2027 | **Ngày kiểm tra:** 10/10/2026

Bài làm gồm phân tích bộ nhớ và cấu trúc dữ liệu, thiết kế lớp cho trung tâm Anh ngữ, cùng chương trình quản lý học viên và giảng viên. Mã nguồn thực thi nằm trong thư mục src; bộ kiểm thử đi kèm kiểm tra các yêu cầu và trường hợp dữ liệu không hợp lệ.

Phần I Tư duy và phân tích mã nguồn

Câu 1.1 Kết quả của năm lệnh print

```
1. list_a[0] = 99
2. list_a[1] = [20, 30, 40]
3. list_a[2] = {'a': 1, 'b': 2}
4. list_b is list_a: True
5. list_c is list_a: False
```

Giải thích cơ chế bộ nhớ

Gán tham chiếu và aliasing: Lệnh `list_b = list_a` không tạo list mới. Hai tên trỏ tới cùng một đối tượng list ngoài. Vì vậy, `list_b[0] = 99` thay phần tử đầu của chính list mà `list_a` tham chiếu; kết quả đầu tiên là 99 và phép kiểm tra `list_b is list_a` cho True.

Sao chép nông: `copy.copy(list_a)` tạo list ngoài mới nhưng sao chép các tham chiếu phần tử. Do đó `list_c is list_a` là False, trong khi `list_c[1] is list_a[1]` và `list_c[2] is list_a[2]` đều là True. Hai list ngoài khác nhau vẫn dùng chung list con và dictionary con.

Biến đổi đối tượng lồng nhau: `list_c[1].append(40)` sửa trực tiếp list con dùng chung thành `[20, 30, 40]`. Tương tự, `list_c[2]['b'] = 2` thêm khóa vào dictionary dùng chung. Các thay đổi này được nhìn thấy qua cả `list_a`, `list_b` và `list_c`.

Tính khả biến: List và dictionary là mutable nên có thể bị sửa tại chỗ. Số nguyên là immutable: phép gán 99 không sửa số 10, mà thay tham chiếu ở ô đầu của list ngoài. Vì `list_c` đã có list ngoài riêng trước phép gán này, `list_c[0]` vẫn bằng 10. Nếu cần sao chép độc lập cả các cấu trúc lồng nhau trong ví dụ, có thể dùng `copy.deepcopy()`.

Tệp `src/phan_tich_bo_nho.py` thực thi nguyên các thao tác trên để đối chiếu trực tiếp năm dòng kết quả.

Câu 1.2 Lựa chọn cấu trúc dữ liệu cho voucher

Vì sao Set và Dictionary Key tìm kiếm nhanh hơn List

Với list, phép kiểm tra `voucher_id in danh_sach` lần lượt so sánh mã cần tìm với các phần tử. Nếu mã nằm cuối hoặc không tồn tại, chương trình có thể phải xét toàn bộ 1.000.000 phần tử. Chi phí tìm kiếm vì vậy là O(n); trường hợp tìm thấy ngay đầu chỉ cần O(1).

Set và dictionary dùng bảng băm. Python tính giá trị hash của mã, dùng nó để xác định vị trí tra cứu trong bảng, rồi kiểm tra ứng viên và xử lý xung đột băm nếu có. Khi bảng băm hoạt động thông thường, tra cứu chỉ cần số bước trung bình gần như không phụ thuộc số mã, nên có độ phức tạp trung bình O(1).

O(1) là đánh giá trung bình theo số phần tử, không phải cam kết cho mọi trường hợp. Khi xung đột băm bất lợi, chi phí có thể tăng tới O(n). Việc tính hash và so sánh chuỗi cũng phụ thuộc độ dài mã; khi so sánh các cấu trúc theo số lượng mã, thường coi độ dài mã là bị chặn.

Cấu trúc	Kiểm tra tồn tại	Điểm phù hợp và hạn chế
List	O(n) nói chung	Giữ thứ tự, cho phép trùng; phải duyệt nhiều phần tử khi tìm mã.
Set	Trung bình O(1), xấu nhất O(n)	Lọc trùng tự động, phù hợp tập mã hợp lệ; không bảo đảm thứ tự duyệt, phần tử phải hashable.
Dictionary key	Trung bình O(1), xấu nhất O(n)	Tra cứu nhanh theo khóa duy nhất, đồng thời gắn được dữ liệu với mỗi mã; cần bộ nhớ cho bảng băm.

Nếu chỉ kiểm tra mã hợp lệ, chọn set chứa các Voucher ID dạng chuỗi. Chi phí tạo tập ban đầu trung bình là O(n), nhưng được bù lại khi hệ thống thực hiện rất nhiều lần kiểm tra. Set và dictionary thường cần bộ nhớ phụ để duy trì bảng băm; đây là đánh đổi để tra cứu nhanh hơn.

Lưu thêm phần trăm giảm giá

Chọn **dictionary**, trong đó key là Voucher ID và value là phần trăm giảm. Cấu trúc này biểu diễn trực tiếp quan hệ mã - mức giảm và cho phép lấy cả thông tin tương ứng bằng một lần tra cứu.

```
vouchers = {"SALE10": 10, "WELCOME20": 20}
ma_nhap = "SALE10"
muc_giam = vouchers.get(ma_nhap)
if muc_giam is None:
    print("Mã không hợp lệ")
else:
    print(f"Giảm {muc_giam}%")
```

`dict.get()` trả về None khi thiếu khóa trong ví dụ này, tránh KeyError. Kiểm tra bằng `is None` vẫn phân biệt được trường hợp một mã có giá trị giảm 0. Các mức giảm nên được kiểm tra trong khoảng 0-100 khi thêm hoặc sửa dữ liệu. Dictionary giữ thứ tự chèn trong Python hiện đại, nhưng việc tra cứu vẫn dựa trên khóa, không dựa vào thứ tự đó.

Phần II Phân tích và thiết kế hướng đối tượng

Câu 2.1 Các lớp và quan hệ

Nguoi là lớp cha của HocVien và GiangVien. Đây là quan hệ kế thừa: học viên và giảng viên đều là người. QuanLyTrungTam giữ tham chiếu đến các HocVien trong dictionary; đây là quan hệ quản lý/chứa đựng, không phải kế thừa từ HocVien.

Lớp	Thuộc tính và phạm vi truy cập	Phương thức và property

Nguoi	ma_id: str, Public;
	ho_ten: str, Public;
	email: str, Public
		__init__(ma_id, ho_ten, email);
		hien_thi_thong_tin();
		__str__()

HocVien kế thừa Nguoi	Các thuộc tính Public của Nguoi;
	danh_sach_khoa: set[str], Public;
	__tong_hoc_phi: int hoặc float, Private
		__init__(ma_id, ho_ten, email);
		dang_ky_khoa_hoc(ten_khoa, hoc_phi);
		property tong_hoc_phi có Getter/Setter;
		hien_thi_thong_tin();
		__eq__(other)

GiangVien kế thừa Nguoi	Các thuộc tính Public của Nguoi; chuyen_mon: str, Public; __luong_theo_gio: int hoặc float, Private	__init__(ma_id, ho_ten, email, chuyen_mon, luong_theo_gio); property luong_theo_gio có Getter/Setter; ghi đè hien_thi_thong_tin()

QuanLyTrungTam	ds_hoc_vien: dict[str, HocVien], Public;
	khóa là ma_id, giá trị là đối tượng HocVien
		__init__();
		them_hoc_vien(hoc_vien);
		tim_hoc_vien(ma_id);
		__len__()

Các phương thức trên là Public. Tiền tố __ kích hoạt name mangling cho học phí và lương, hạn chế truy cập nhầm từ bên ngoài. Tiền tố _ biểu thị quy ước nội bộ/Protected; thiết kế này chọn Private cho hai thuộc tính tiền. Getter cho phép đọc; Setter kiểm tra số hữu hạn và không âm trước khi cập nhật. Name mangling không phải cơ chế bảo mật tuyệt đối.

Câu 2.2 Đóng gói và định nghĩa chồng

`super().hien_thi_thong_tin()` gọi phần hiển thị chung của lớp cha theo thứ tự tra cứu phương thức. Trong cây kế thừa này, đó là Nguoi. GiangVien lấy thông tin mã, họ tên, email đã được định dạng rồi nối thêm chuyên môn và lương theo giờ. Nhờ vậy, lớp con mở rộng hành vi mà vẫn giữ phần thông tin chung.

Nếu không dùng super(), có thể chép lại đoạn định dạng của Nguoi, nhưng mã sẽ bị trùng. Khi cần đổi cách hiển thị email hoặc thêm thông tin chung, phải sửa nhiều nơi; dễ bỏ sót, tạo kết quả không nhất quán và tăng công bảo trì. Nếu không gọi super() cũng không viết lại phần chung, thông tin mã, tên hoặc email có thể bị mất. Không dùng super() không tự động gây lỗi chạy; vấn đề chính là nguy cơ thiếu dữ liệu và trùng lặp mã.

Hai lớp con cũng dùng `super().__init__()` để khởi tạo các thuộc tính chung. Việc trả về chuỗi từ `hien_thi_thong_tin()` giúp cùng một vòng lặp có thể gọi phương thức trên HocVien và GiangVien, minh họa tính đa hình.

Phần III Lập trình và kiểm chứng hệ thống

Cách hiện thực các yêu cầu

Nguoi có constructor đúng ba thông tin và `__str__()` trả về chuỗi dạng `[HV01] Nguyen Van A`. HocVien khởi tạo `danh_sach_khoa = set()` và `__tong_hoc_phi = 0` cho từng đối tượng riêng. Property `tong_hoc_phi` hỗ trợ đúng cách đọc `hv1.tong_hoc_phi` trong test case của đề.

Trong `dang_ky_khoa_hoc()`, học phí phải là số hữu hạn và lớn hơn 0. Với học phí bằng 0 hoặc âm, chương trình phát sinh chính xác `ValueError("Học phí phải lớn hơn 0!")`. Tên khóa phải là chuỗi không rỗng; khoảng trắng hai đầu được loại bỏ.

Set chỉ ngăn phần tử trùng, không tự ngăn cộng tiền trùng. Vì vậy, chương trình kiểm tra khóa đã tồn tại trước khi cộng học phí: đăng ký mới trả True; đăng ký trùng trả False và không thay đổi học phí. Hai lần đăng ký Python Basic chỉ tạo một khóa và tổng học phí 1.500.000 VNĐ. Tên khóa được so sánh phân biệt hoa/thường. Dữ liệu không hợp lệ được từ chối trước khi thay đổi trạng thái.

QuanLyTrungTam dùng dictionary theo ma_id, `tim_hoc_vien()` gọi get() và trả None khi không tìm thấy. `__len__()` trả số học viên. Thêm trùng mã bị từ chối để không ghi đè người cũ. `HocVien.__eq__()` so sánh ma_id khi cả hai là HocVien, trả NotImplemented cho kiểu khác. Học viên được lưu làm giá trị dictionary; mã chuỗi là khóa.

Học phí tổng và lương giờ được đọc/ghi qua property. Setter cho phép số không âm; học phí của một lần đăng ký mới phải dương. Setter tổng hỗ trợ điều chỉnh tổng tiền trực tiếp theo ràng buộc này, chưa mô hình hóa sổ giao dịch hay quyền người cập nhật. Trong luồng đăng ký, tổng học phí được cộng qua setter; trạng thái cũ được giữ nếu tổng mới không hợp lệ.

Kiểm thử và kết quả

Đã chạy hai chương trình minh họa bằng Python 3.14.4 và bộ unittest gồm **17 test, tất cả đạt**. Mã nghiệp vụ chỉ dùng thư viện chuẩn, không cần cài thêm gói.

Nhóm kiểm tra	Kết quả đối chiếu
Bộ nhớ câu 1.1	Đúng cả năm dòng print.
Đăng ký trùng và khóa mới	Một khóa Python Basic: 1.500.000; thêm IELTS Foundation: 3.500.000 VNĐ.
Ràng buộc dữ liệu	Từ chối học phí không dương, tên rỗng, sai kiểu, NaN/vô hạn và tổng không hợp lệ; không cập nhật nửa chừng.
Đóng gói và kế thừa	Getter/Setter hợp lệ, thông tin cha được giữ, dữ liệu riêng từng đối tượng.
Dictionary và toán tử	Tra cứu trả đúng đối tượng hoặc None; không ghi đè mã trùng; len và == đúng theo yêu cầu.

Tệp đi kèm

Mã lớp: `src/quan_ly_trung_tam.py`. Minh họa: `src/main.py` và `src/phan_tich_bo_nho.py`. Kiểm thử: `tests/test_quan_ly_trung_tam.py`. Hướng dẫn chạy và cấu trúc gói nộp được ghi trong README.md.